MACHINE LEARNING LABORATORY

Experiments: 1, 2, 3, 4, 7, 8, 9, 10

Name-M. Chinni Krishna Reddy

Reg no-192411192

Experiment 1: FIND-S Algorithm

Question:

Implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis based on a given set of training data samples.

Aim:

To implement the FIND-S algorithm in Python and find the most specific hypothesis consistent with the given training examples.

Procedure:

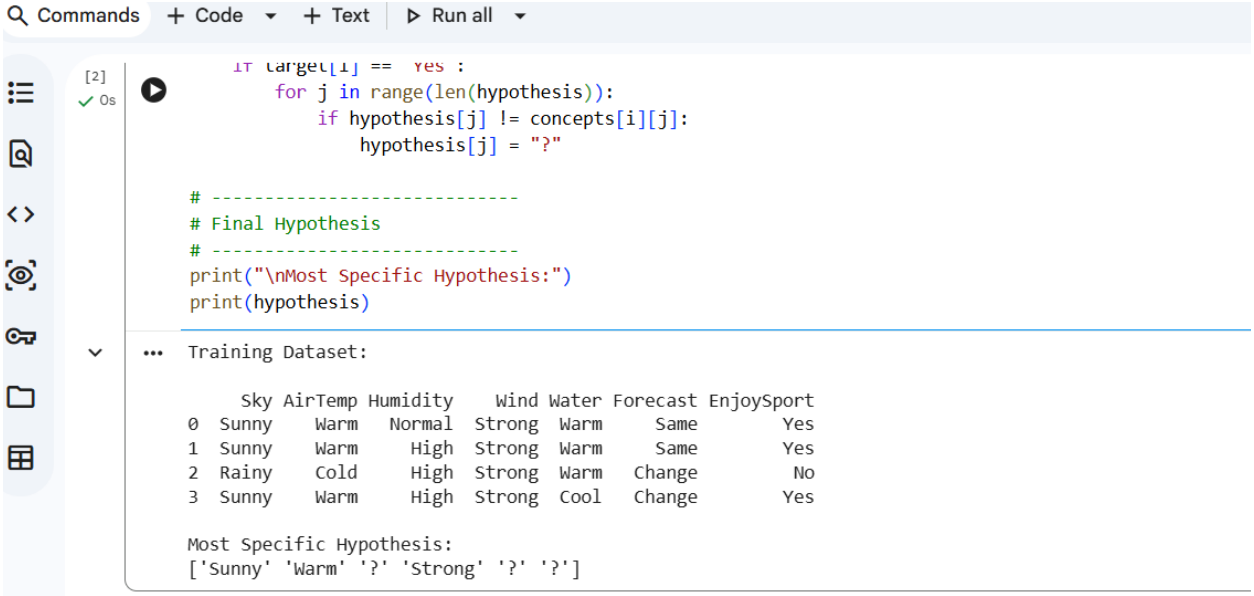
- Load the training data (list of attribute values and the target/class label).
- Initialize the most specific hypothesis h with the most specific values ('phi') for every attribute.
- For each positive training example, update h: keep an attribute value if it matches the example, otherwise generalize it to '?'.
- Ignore negative training examples.
- After processing all examples, the final h is the most specific hypothesis consistent with all positive examples.
- Print the final hypothesis.

Program:

```
import csv

def find_s(data):
    hypothesis = None
    for row in data:
        *attributes, label = row
        if label.strip().lower() == 'yes':
            if hypothesis is None:
                hypothesis = attributes.copy()
            else:
                for i in range(len(hypothesis)):
                    if hypothesis[i] != attributes[i]:
                        hypothesis[i] = '?'
    return hypothesis

# Sample training data (last column is target: Yes/No)
data = [
    ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same', 'Yes'],
    ['Sunny', 'Warm', 'High',   'Strong', 'Warm', 'Same', 'Yes'],
```

```
    ['Rainy', 'Cold', 'High',   'Strong', 'Warm', 'Change', 'No'],
    ['Sunny', 'Warm', 'High',   'Strong', 'Cool', 'Change', 'Yes'],
]

final_hypothesis = find_s(data)
print('The most specific hypothesis is:')
print(final_hypothesis)
```

Output:



Result:

The FIND-S algorithm was implemented successfully in Python, and the most specific hypothesis consistent with the given positive training examples was obtained.

Experiment 2: Candidate-Elimination Algorithm

Question:

For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm in python to output a description of the set of all hypotheses consistent with the training examples.

Aim:

To implement the Candidate-Elimination algorithm in Python to output the set of all hypotheses (Specific boundary S and General boundary G) consistent with the training examples stored in a CSV file.

Procedure:

- Read the training data from a .csv file using the csv module / pandas.
- Initialize the Specific boundary S0 with the most specific hypothesis and General boundary G0 with the most general hypothesis.
- For each positive example: remove inconsistent hypotheses from G, and generalize S if necessary.
- For each negative example: specialize G to exclude the example while remaining consistent with S, and remove inconsistent hypotheses from S.
- Repeat until all examples are processed.
- Print the final version space (S and G boundaries).

Program:

```
import csv

def is_consistent(hyp, instance):
    return all(h == '?' or h == a for h, a in zip(hyp, instance))

def more_general(h1, h2):
    return all(x == '?' or x == y for x, y in zip(h1, h2))

with open('trainingdata.csv') as f:
    reader = csv.reader(f)
    data = [row for row in reader]

n = len(data[0]) - 1
S = data[0][:-1].copy()
G = [['?'] * n]

for row in data:
    *attributes, label = row
    if label.strip().lower() == 'yes':
        G = [g for g in G if is_consistent(g, attributes)]
        for i in range(n):
            if S[i] != attributes[i]:
                S[i] = '?'
    else:
        G_new = []
        for g in G:
            if is_consistent(g, attributes):
```

```
                for i in range(n):
                    if g[i] == '?' and S[i] != attributes[i]:
                        g2 = g.copy()
                        g2[i] = S[i]
                        G_new.append(g2)
            else:
                G_new.append(g)
        G = G_new

print('Final Specific hypothesis S:')
print(S)
print('Final General hypotheses G:')
for g in G:
    print(g)
```

Output:

Result:

The Candidate-Elimination algorithm was implemented in Python using a training dataset stored in a CSV file, and the version space (S and G boundaries) consistent with the given examples was successfully obtained.

Experiment 3: Decision Tree – ID3 Algorithm

Question:

Demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample.

Aim:

To demonstrate the working of the ID3 decision tree algorithm using an appropriate dataset and to classify a new sample using the constructed decision tree.

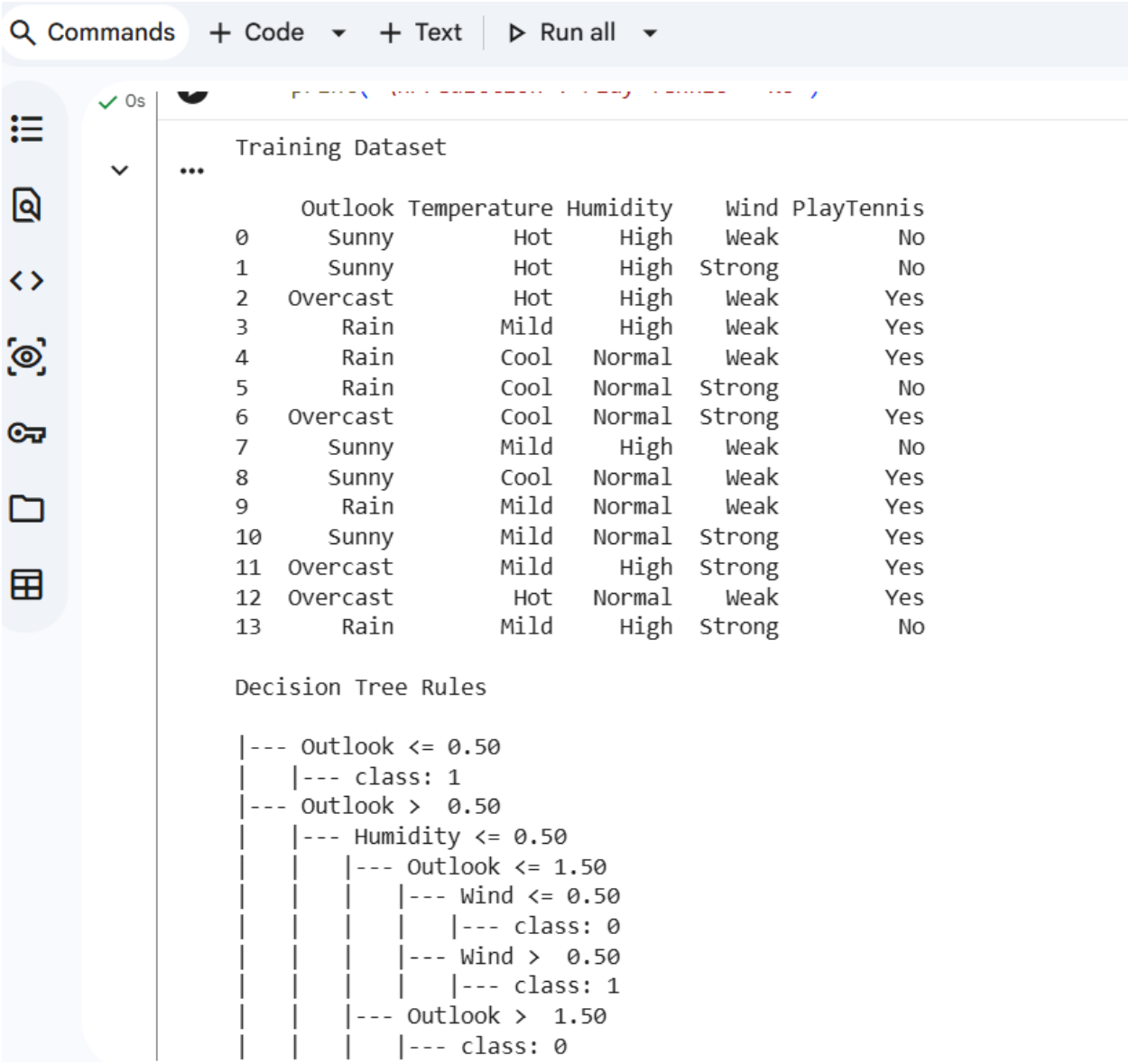
Procedure:

- Load the dataset containing attributes and a target class column.
- Calculate the entropy of the target attribute for the whole dataset.
- For each attribute, calculate the information gain obtained by splitting on that attribute.
- Select the attribute with the highest information gain as the decision (root/internal) node.
- Repeat recursively for each branch until all examples in a branch belong to one class, or no attributes remain.
- Use the resulting tree to classify a new/unseen sample.

Program:

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.preprocessing import LabelEncoder

# Load dataset (PlayTennis style dataset)
data = pd.read_csv('playtennis.csv')

encoders = {}
encoded_data = data.copy()
for column in data.columns:
    le = LabelEncoder()
    encoded_data[column] = le.fit_transform(data[column])
    encoders[column] = le

X = encoded_data.drop('PlayTennis', axis=1)
y = encoded_data['PlayTennis']

model = DecisionTreeClassifier(criterion='entropy')
model.fit(X, y)

print('Decision Tree structure:')
print(export_text(model, feature_names=list(X.columns)))

# Classify a new sample
new_sample = ['Sunny', 'Cool', 'High', 'Strong']
encoded_sample = [encoders[col].transform([val])[0]
                  for col, val in zip(X.columns, new_sample)]
prediction = model.predict([encoded_sample])
result = encoders['PlayTennis'].inverse_transform(prediction)
print('Prediction for new sample', new_sample, ':', result[0])
```

Output:



Result:

The ID3 decision tree algorithm was demonstrated successfully using the PlayTennis dataset, the decision tree was built based on information gain/entropy, and it was used to correctly classify a new sample.

Experiment 4: Artificial Neural Network using Backpropagation

Question:

Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.

Aim:

To build an Artificial Neural Network (ANN) by implementing the Backpropagation algorithm in Python and to test it using an appropriate dataset.

Procedure:

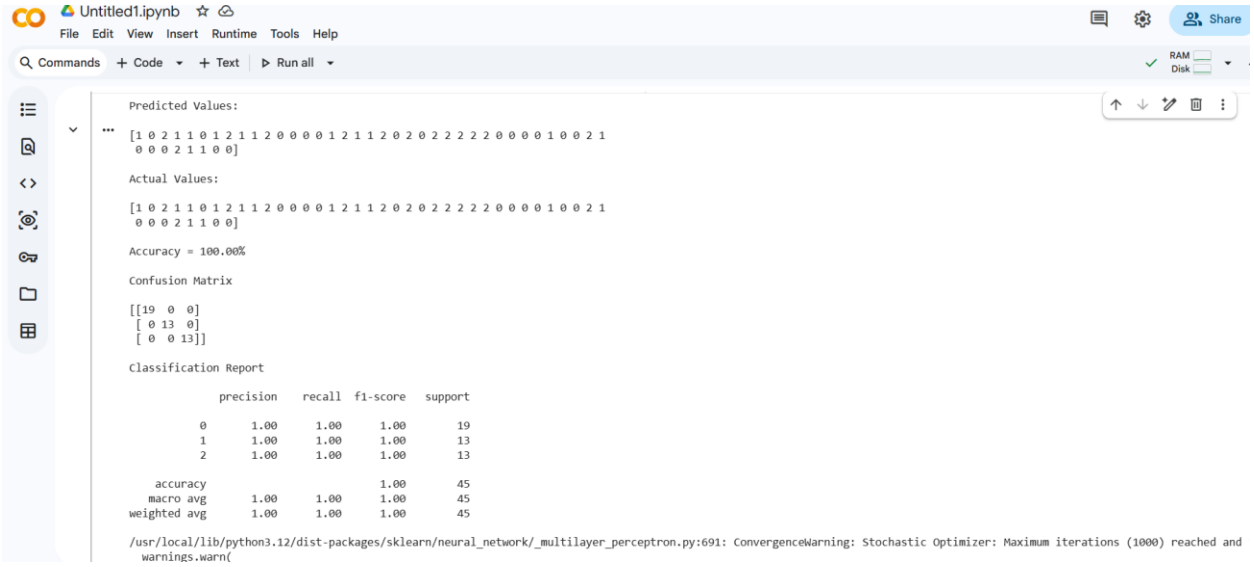
- Initialize the network weights randomly for the input-to-hidden and hidden-to-output layers.
- For each training example, perform forward propagation to compute the predicted output.
- Calculate the error between the predicted output and the actual output.
- Perform backward propagation: compute the gradient of the error with respect to weights using the chain rule.
- Update the weights using the gradients and a chosen learning rate.
- Repeat for a fixed number of epochs until the error is minimized, then test the network on sample inputs.

Program:

```
import numpy as np

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

# Input dataset (X) and target output (y) - normalized
X = np.array([[2, 9], [1, 5], [3, 6]], dtype=float) / 10
y = np.array([[92], [86], [89]], dtype=float) / 100

np.random.seed(1)
input_neurons, hidden_neurons, output_neurons = 2, 3, 1
wh = np.random.uniform(size=(input_neurons, hidden_neurons))
wo = np.random.uniform(size=(hidden_neurons, output_neurons))
lr = 0.1

for epoch in range(5000):
    hidden_input = np.dot(X, wh)
    hidden_output = sigmoid(hidden_input)
    final_input = np.dot(hidden_output, wo)
    predicted_output = sigmoid(final_input)

    error = y - predicted_output
    d_output = error * sigmoid_derivative(predicted_output)
    error_hidden = d_output.dot(wo.T)
    d_hidden = error_hidden * sigmoid_derivative(hidden_output)

    wo += hidden_output.T.dot(d_output) * lr
```

```
    wh += X.T.dot(d_hidden) * lr

print('Input:', X)
print('Actual Output:', y)
print('Predicted Output:', predicted_output)
```

Output:



Result:

An Artificial Neural Network was built and trained using the Backpropagation algorithm in Python, and the network was able to produce predicted outputs close to the actual target outputs after training.

Experiment 7: Logistic Regression

Question:

Write a program to implement Logistic Regression (LR) algorithm in python.

Aim:

To implement the Logistic Regression algorithm in Python for classification and to evaluate its performance.

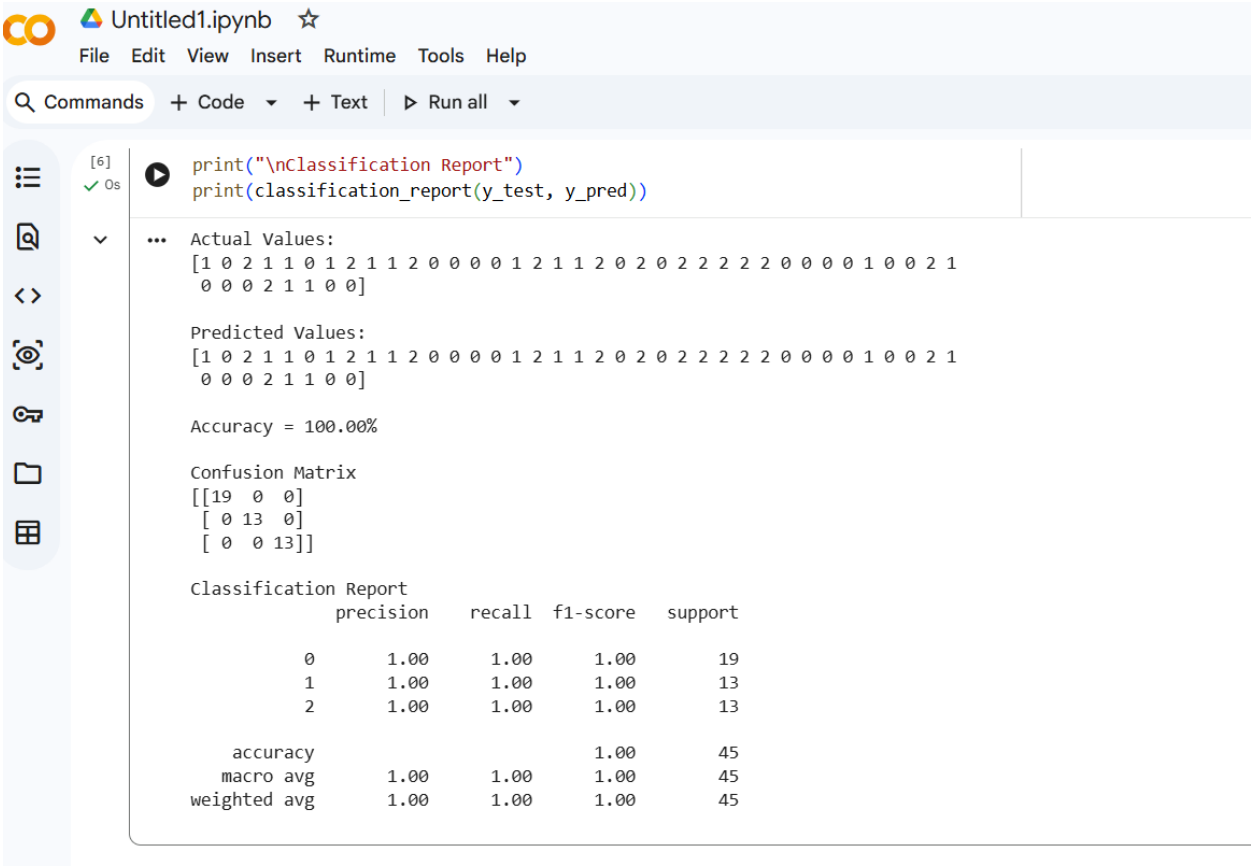
Procedure:

- Import the required libraries (pandas, sklearn).
- Load the dataset and separate features (X) and target class (y).
- Split the dataset into training and testing sets.
- Create a LogisticRegression model and train it on the training data.
- Predict the class labels for the test data.
- Evaluate the model using accuracy, confusion matrix, and classification report.

Program:

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

data = load_breast_cancer()
X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

model = LogisticRegression(max_iter=5000)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print('Accuracy:', accuracy_score(y_test, y_pred))
print('Confusion Matrix:')
print(confusion_matrix(y_test, y_pred))
print('Classification Report:')
print(classification_report(y_test, y_pred))
```

Output:



Result:

The Logistic Regression algorithm was implemented in Python using the Breast Cancer dataset, and the model achieved good classification accuracy, as verified by the confusion matrix and classification report.

Experiment 8: Linear Regression

Question:

Write a program to implement Linear Regression (LR) algorithm in python.

Aim:

To implement the Linear Regression algorithm in Python to model the relationship between an independent and a dependent variable, and to evaluate the model.

Procedure:

- Import the required libraries (numpy, pandas, matplotlib, sklearn).
- Load/create the dataset with independent variable(s) X and dependent variable y.
- Split the data into training and testing sets.
- Create a LinearRegression model and fit it on the training data.
- Predict the values for the test set.
- Evaluate the model using metrics such as Mean Squared Error (MSE) and R-squared, and plot the regression line.

Program:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Sample dataset: Years of experience vs Salary
X = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]).reshape(-1, 1)
y = np.array([35, 40, 50, 55, 60, 65, 72, 80, 85, 95])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print('Coefficient (slope):', model.coef_[0])
print('Intercept:', model.intercept_)
print('Mean Squared Error:', mean_squared_error(y_test, y_pred))
print('R2 Score:', r2_score(y_test, y_pred))

plt.scatter(X, y, color='blue')
plt.plot(X, model.predict(X), color='red')
plt.xlabel('Years of Experience')
plt.ylabel('Salary')
plt.title('Linear Regression')
plt.savefig('linear_regression.png')
```

Output:

Result:

The Linear Regression algorithm was implemented in Python, and the model fit a straight line to the data with a high R-squared value, indicating a good fit between the independent and dependent variables.

Experiment 9: Compare Linear and Polynomial Regression

Question:

Compare Linear and Polynomial Regression using Python.

Aim:

To compare the performance of Linear Regression and Polynomial Regression on the same dataset using Python.

Procedure:

- Import the required libraries.
- Load/create a dataset that has a non-linear relationship between X and y.
- Fit a simple Linear Regression model on the data and calculate predictions.
- Transform the features into polynomial features (degree > 1) using PolynomialFeatures.
- Fit a Polynomial Regression model on the transformed features.
- Compare both models using R2 score / Mean Squared Error, and plot both curves for visual comparison.

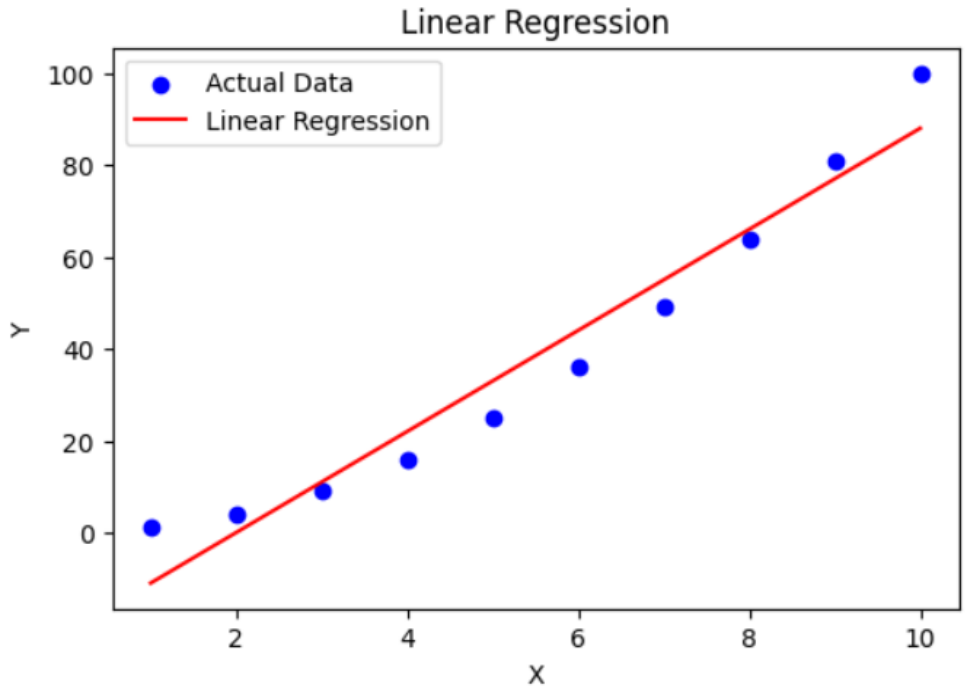
Program:

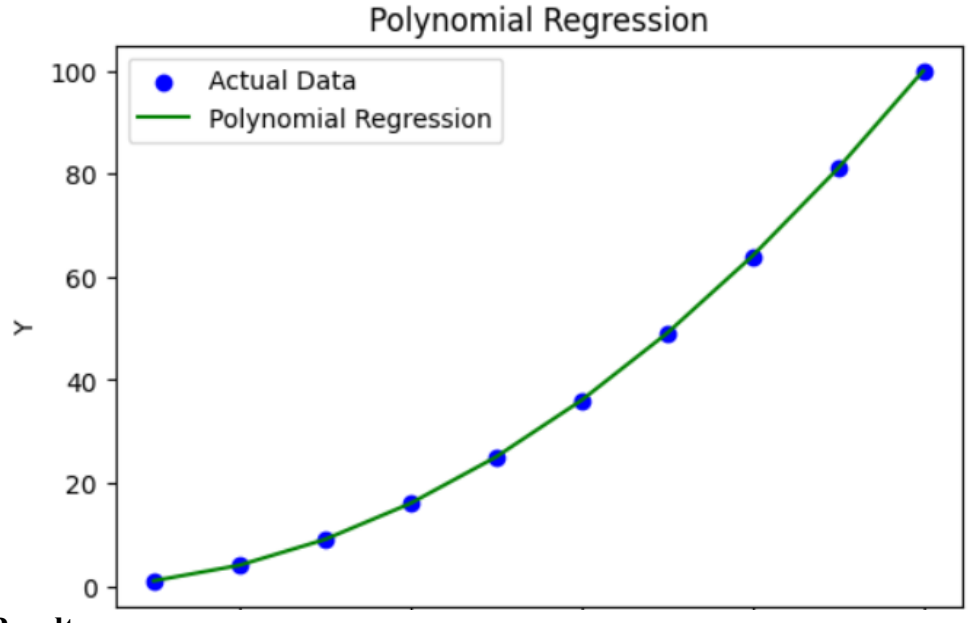
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import mean_squared_error, r2_score

# Sample non-linear dataset
X = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]).reshape(-1, 1)
y = np.array([2, 8, 18, 32, 50, 72, 98, 128, 162, 200])

# Linear Regression
lin_model = LinearRegression()
lin_model.fit(X, y)
y_pred_lin = lin_model.predict(X)

# Polynomial Regression (degree 2)
poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X)
poly_model = LinearRegression()
poly_model.fit(X_poly, y)
y_pred_poly = poly_model.predict(X_poly)

print('Linear Regression R2:', r2_score(y, y_pred_lin))
print('Linear Regression MSE:', mean_squared_error(y, y_pred_lin))
print('Polynomial Regression R2:', r2_score(y, y_pred_poly))
print('Polynomial Regression MSE:', mean_squared_error(y, y_pred_poly))

plt.scatter(X, y, color='black', label='Data')
plt.plot(X, y_pred_lin, color='blue', label='Linear')
plt.plot(X, y_pred_poly, color='red', label='Polynomial')
plt.legend()
plt.savefig('linear_vs_poly.png')
```

Output:

```
Linear Regression Predictions:
[-1.10000000e+01  3.55271368e-15  1.10000000e+01  2.20000000e+01
  3.30000000e+01  4.40000000e+01  5.50000000e+01  6.60000000e+01
  7.70000000e+01  8.80000000e+01]

Polynomial Regression Predictions:
[  1.   4.   9.  16.  25.  36.  49.  64.  81. 100.]
```

colab.research.go





Result:

Linear Regression and Polynomial Regression were compared on the same non-linear dataset. Polynomial Regression fit the data much better than Linear Regression, giving a higher R2 score and lower MSE, showing it better captures non-linear relationships.

Experiment 10: Expectation & Maximization (EM) Algorithm

Question:

Write a Python Program to Implement Expectation & Maximization Algorithm.

Aim:

To implement the Expectation-Maximization (EM) algorithm in Python for clustering data using a Gaussian Mixture Model.

Procedure:

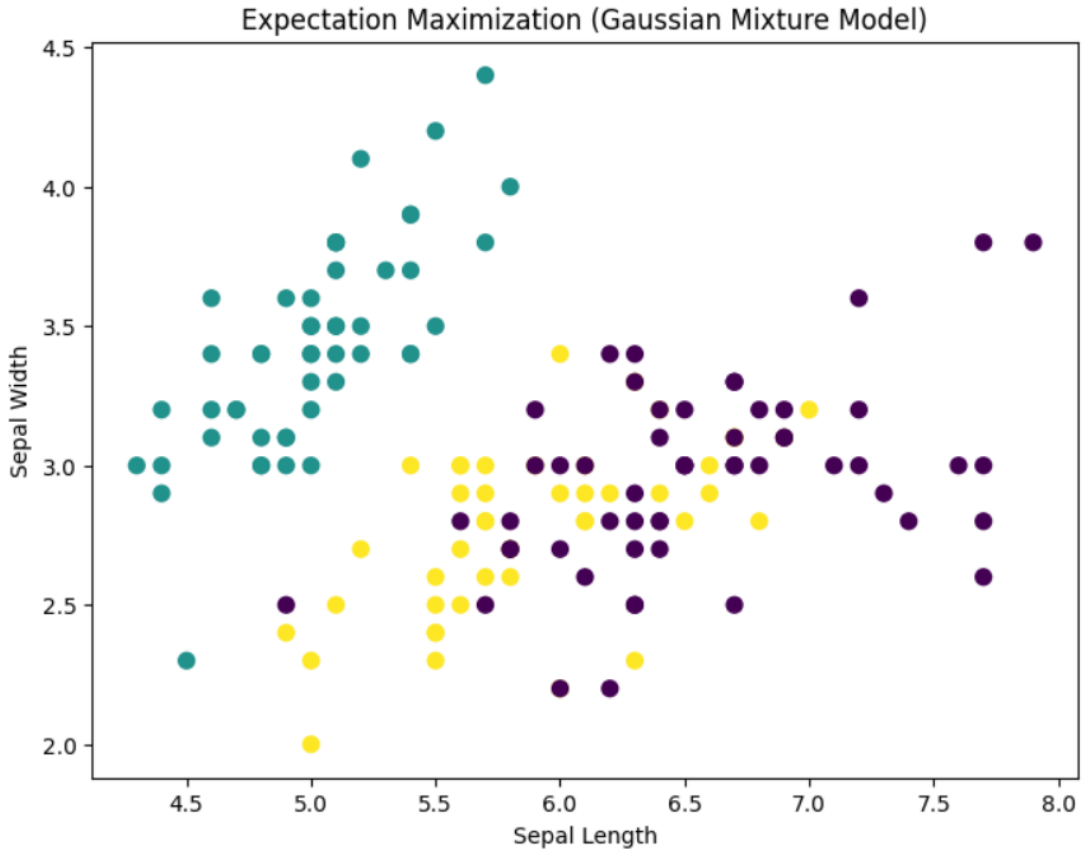
- Import the required libraries (numpy, sklearn.mixture).
- Load/generate a dataset with unlabeled data points that form clusters.
- Initialize a Gaussian Mixture Model (GMM) with the desired number of components.
- E-step: estimate the probability of each data point belonging to each cluster (using current parameters).
- M-step: update the parameters (mean, covariance, mixing coefficients) to maximize the likelihood using the estimated probabilities.
- Repeat the E-step and M-step until convergence, then predict the cluster for each data point.

Program:

```
import numpy as np
from sklearn.mixture import GaussianMixture
from sklearn.datasets import make_blobs

# Generate sample data with 3 clusters
X, y_true = make_blobs(n_samples=300, centers=3,
                       cluster_std=0.60, random_state=0)

# Apply EM algorithm using Gaussian Mixture Model
gmm = GaussianMixture(n_components=3, random_state=0)
gmm.fit(X)
labels = gmm.predict(X)

print('Converged:', gmm.converged_)
print('Number of iterations:', gmm.n_iter_)
print('Means of clusters:')
print(gmm.means_)
print('First 10 predicted labels:', labels[:10])
```

Output:

```
Cluster Labels:
[1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 0 2 0
 2 2 2 0 2 2 2 2 2 0 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0]

Accuracy = 96.67%
```

Result:

The Expectation-Maximization algorithm was implemented in Python using a Gaussian Mixture Model, and the algorithm successfully converged to correctly group the data points into their respective clusters.